

**INSTITUTO TECNOLÓGICO SUPERIOR DE CALKINÍ,
EN EL ESTADO DE CAMPECHE**

ingeniería en Sistemas Computacionales

Nombre Completo & Matricula:

Fernando Jesus Colli Cach 9033

TOPICOS AVANZADOS DE PROGRAMACION

Maestro: JOSE MANUEL LUIS TURRIZA

**PRÁCTICA No. 2- Generación de Aplicaciones con UI y
eventos**

4 SEMESTRE

GRUPO: A

	Dirección Académica	Código: CPE-FO-02-03
		Revisión: 1
	MANUAL DE PRÁCTICAS	Página: 5 de 27

PRÁCTICA No. 2- Generación de Aplicaciones con UI y eventos

-INTRODUCCIÓN

Programación orientada a eventos.

La programación dirigida por eventos es un paradigma de la programación en el que tanto la estructura como la ejecución de los programas van determinados por los sucesos que ocurran en el sistema, definidos por el usuario o que ellos mismos provoquen.

Para entender la programación dirigida por eventos, podemos oponerla a lo que no es: mientras en la programación secuencial (o estructurada) es el programador el que define cuál va a ser el flujo del programa, en la programación dirigida por eventos será el propio usuario o lo que sea que esté accionando el programa, el que dirija el flujo del programa. Aunque en la programación secuencial puede haber intervención de un agente externo al programa, estas intervenciones ocurrirán cuando el programador lo haya determinado, y no en cualquier momento como puede ser en el caso de la programación dirigida por eventos.

El creador de un programa dirigido por eventos debe definir los eventos que manejarán su programa y las acciones que se realizarán al producirse cada uno de ellos, lo que se conoce como el administrador del evento. Los eventos soportados estarán determinados por el lenguaje de programación utilizado, por el sistema operativo e incluso por eventos creados por el mismo programador.

En la programación dirigida por eventos, al comenzar la ejecución del programa se llevarán a cabo las inicializaciones y demás código inicial y a continuación el programa quedará bloqueado hasta que se produzca algún evento. Cuando alguno de los eventos esperados por el programa tenga lugar, el programa pasará a ejecutar el código del correspondiente administrador de evento. Por ejemplo, si el evento consiste en que el usuario ha hecho clic en el botón de play de un reproductor de películas, se ejecutará el código del administrador de evento, que será el que haga que la película se muestre por pantalla.

Un ejemplo claro lo tenemos en los sistemas de programación Lógico y Visual Basic, en los que a cada elemento del programa (objetos, controles, etcétera) se le asignan una serie de eventos que generará dicho elemento, como la pulsación de un botón del ratón sobre él o el redibujado del control.

La programación dirigida por eventos es la base de lo que llamamos interfaz de usuario, aunque puede emplearse también para desarrollar interfaces entre componentes de Software o módulos de núcleos.

	Dirección Académica	Código: CPE-FO-02-03
		Revisión: 1
	MANUAL DE PRÁCTICAS	Página: 6 de 27

-OBJETIVO

El estudiante deberá de identificar los elementos de programación de eventos además del desarrollo de aplicaciones de software, en base al uso de diagramas uml y programación en el lenguaje Java.

LUGAR

AULA

-SEMANA DE EJECUCIÓN

SEMANA DOS (Parcial 1)

- MATERIAL Y EQUIPO

- Sistema Operativo
- Procesador de Textos
- Software para el desarrollo de aplicaciones “eclipse”.
- Cañón
- Plumones • Pizarrón.
- Equipos de cómputo para todos los estudiantes de la asignatura.

- DESARROLLO DE LA PRÁCTICA

	Dirección Académica	Código: CPE-FO-02-03
		Revisión: 1
	MANUAL DE PRÁCTICAS	Página: 7 de 27

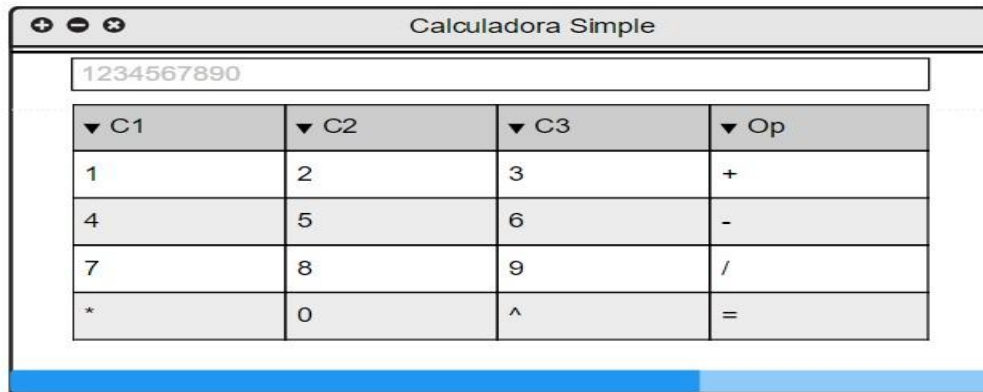


Figura 1. Calculadora Simple

Se desea manipular un conjunto de elementos para implementar la funcionalidad de las operaciones de una calculadora simple de acuerdo a lo siguiente:

1.- Se requiere realizar una aplicación de escritorio que realice los cálculos simples de números enteros de hasta 20 dígitos. En ella solamente se podrá introducir un número por vez seguido de una operación matemática; las operaciones matemáticas tienen prioridades para realizarse que deben aplicarse en la calculadora; el resultado final deberá mostrarse cuando se presione el operador =. En el caso de que la operación pueda realizarse se debe mostrar el resultado en el mismo espacio donde se introducen los números, en caso de algún error, éste se debe mostrar en el mismo espacio. Los errores que se deben considerar son los de división entre cero y desborde de memoria en caso de exceder los 20 dígitos. En esta tarea se deben escribir los casos de uso de la aplicación y los diagramas de caso de uso. También se debe representar la aplicación al menos en los diagramas UML de clases, de secuencia y de casos de uso.

2.- Una vez que se ha diseñado la aplicación se deberá desarrollar la aplicación dividida en dos partes: la interfaz de usuario y la lógica.

2.1 La interfaz gráfica de usuario (GUI por sus siglas en inglés) debe ser construido a partir del siguiente esqueleto:

```
public class CalculadoraGUI extends JFrame
{
    // Los atributos pueden ser componentes de la interfaz como
    // botones y campos de texto
    // En el constructor podemos tener un parámetro que sea el
```

	Dirección Académica	Código: CPE-FO-02-03
		Revisión: 1
	MANUAL DE PRÁCTICAS	Página: 8 de 27

```
//título que le queremos dar al marco, dicho parámetro se lo
//pasaremos al constructor
public CalculadoraGUI (String nombre)
{
    super (nombre);
    // A continuación se definen y añaden las componentes de la
    //interfaz
}
}
```

En esta etapa se deben inicializar cada uno de los elementos que para esta práctica serán 16 botones distribuidos dentro de un grid, y un campo de texto para introducir los números y mostrar los resultados. Configura los elementos de la siguiente manera:

Ventana principal con título “Calculadora Simple” de tamaño en pixeles de 500 x 400.

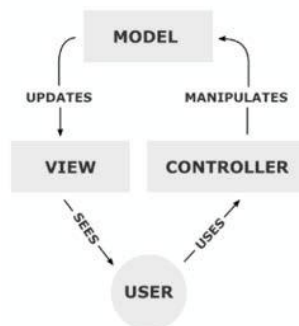
Campo de Texto con el texto inicial de “1234567890” en color gris.

16 botones en formato plano como se muestra en la figura 1, cuyo texto sean los números del 1 al 9, 0, +, -, *, / y ^, como los números naturales, el cero, suma, resta, multiplicación, división y potencia respectivamente. La primera fila contiene etiquetas que hacen referencia a las columnas C1, C2, C3 y Op., incluye éstos con ese texto.

Utiliza para la distribución general de la calculadora un administrador de componentes de tipo BorderLayout y para paneles internos un administrador GridLayout.

Implementa la visibilidad de la ventana en la creación de instancia de la clase. Al finalizar ejecuta tu aplicación y evidencie su resultado con una imagen de su escritorio.

2.2 La lógica de la aplicación deberá estar diseñada de acuerdo a un patrón. En nuestro caso utilizaremos el Modelo-Vista-Controlador (MVC) cuya vista ha sido desarrollada en el punto 2.1 y por lo que deberemos construir el Controlador y el Modelo de acuerdo al diagrama de la figura 2.



	Dirección Académica	Código: CPE-FO-02-03
		Revisión: 1
	MANUAL DE PRÁCTICAS	Página: 9 de 27

Figura 2. Patrón Modelo-Vista-Controlador (MVC)

2.2.1 El Modelo. Estará compuesto por la implementación de la lógica de negocio en la que se implementarán las operaciones tales como las CRUD o alguna otra necesaria como por ejemplo validarNumero; las clases de tipo Value Object(VO) en el que representamos los objetos con sus atributos y sus métodos set y get; y las clases de tipo data access object (DAO) que serán implementados como clases Hlp con un método action de cada tipo de operación de acceso a los datos.

2.2.2 El Controlador. Esta parte del patrón define la lógica de administración del sistema, establece la conexión entre la vista y el modelo.

Para la implementación del controlador se creará una clase Principal única y una clase Controlador por cada interfaz gráfica de la aplicación. Estos últimos deben contener la relación entre el modelo y las vistas.

En la clase Principal implementar el método main donde se instanciarán cada una de las vistas, del modelo y del controlador.

En la clase Controlador se deben tener instancias de la vista y de la lógica del negocio y métodos que permitan realizar las acciones definidas en las dos partes, como por ejemplo sumar o multiplicar.

Para terminar el armado del patrón, se modificará la vista incluyendo la implementación del ActionListener a la clase para obtener los datos de la interfaz gráfica y vaciarlos al objeto y viceversa a través de la implementación de los métodos dataToUserInterface y userInterfaceToData así como el método actionPerformed.

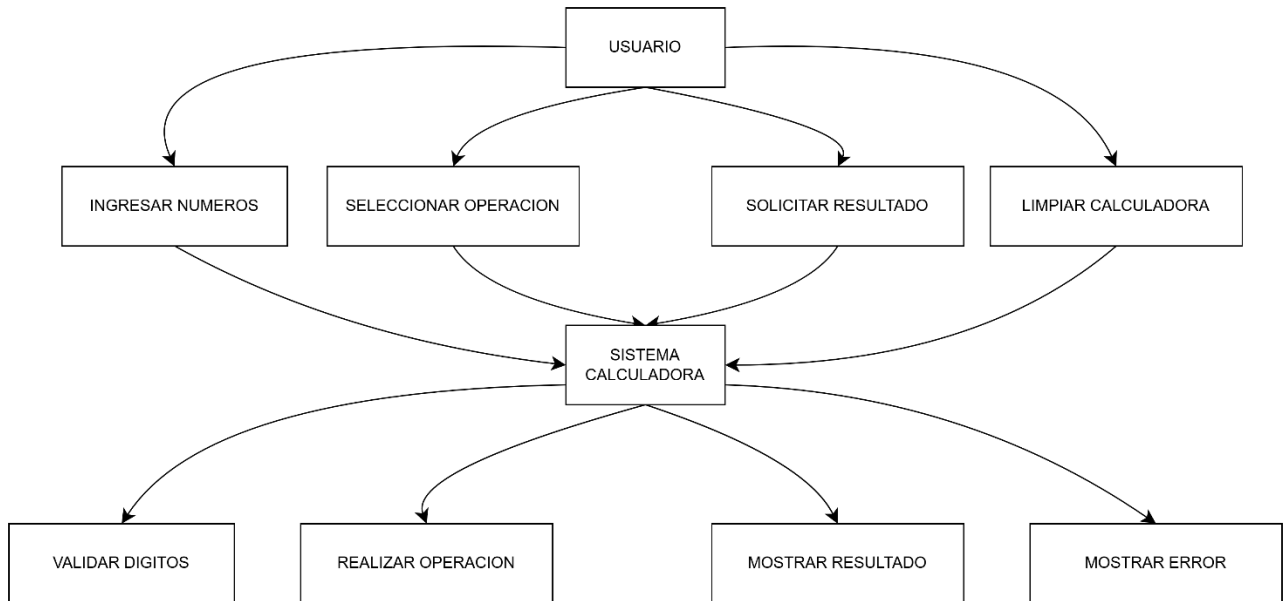
3. Realiza pruebas del prototipo con datos límite de acuerdo al planteamiento del problema, evidencie esta etapa con el diseño de una prueba con el siguiente formato:

Nombre de la prueba:	
Descripción:	
Datos de entrada de prueba:	
Datos de salida esperado:	

4. Realice un informe de la práctica en donde se muestren los resultados y presente una discusión analizando los beneficios del patrón en la aplicación desarrollada.

DIAGRAMAS UML

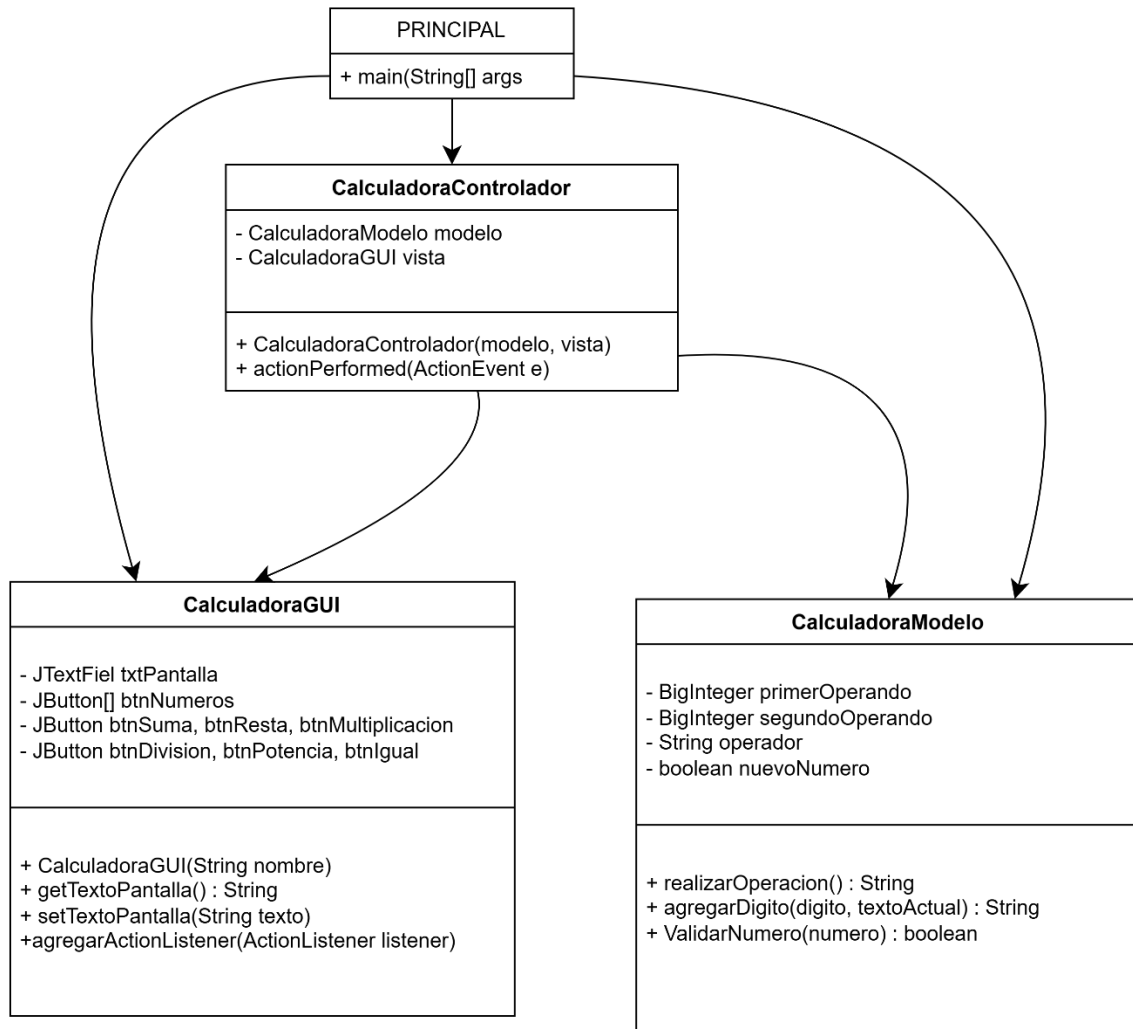
Diagrama de Casos de Uso



Caso de Uso	Ingresar Números
Actor	Usuario
Descripción	El usuario ingresa dígitos para formar un número
Precondición	La calculadora está encendida
Flujo Normal	1. Usuario presiona botón numérico 2. El dígito se muestra en pantalla 3. Se acumulan hasta 20 dígitos
Flujo Alternativo	Si se exceden 20 dígitos, muestra "Error: Desborde"
Caso de Uso	Realizar Operación
Actor	Usuario
Descripción	El usuario selecciona una operación matemática
Precondición	Hay un número ingresado
Flujo Normal	1. Usuario ingresa primer número 2. Selecciona operador (+, -, *, /, ^) 3. Ingresar segundo número 4. Presiona "=" para ver resultado
Flujo Alternativo	Si división entre cero, muestra "Error: Div/0"

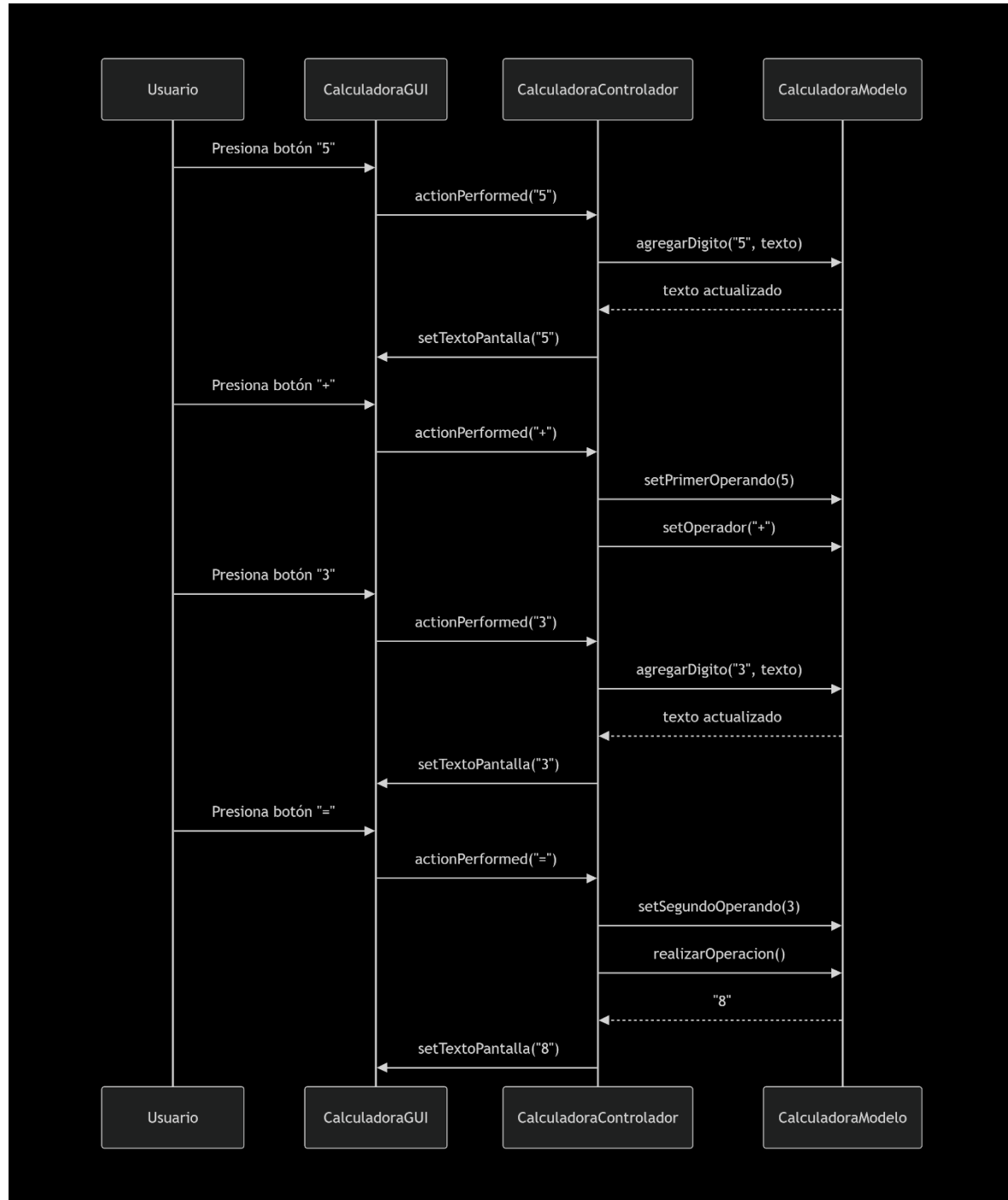
	Dirección Académica	Código: CPE-FO-02-03
		Revisión: 1
	MANUAL DE PRÁCTICAS	Página: 11 de 27

DIAGRAMA DE CLASES



	Dirección Académica	Código: CPE-FO-02-03
		Revisión: 1
	MANUAL DE PRÁCTICAS	Página: 12 de 27

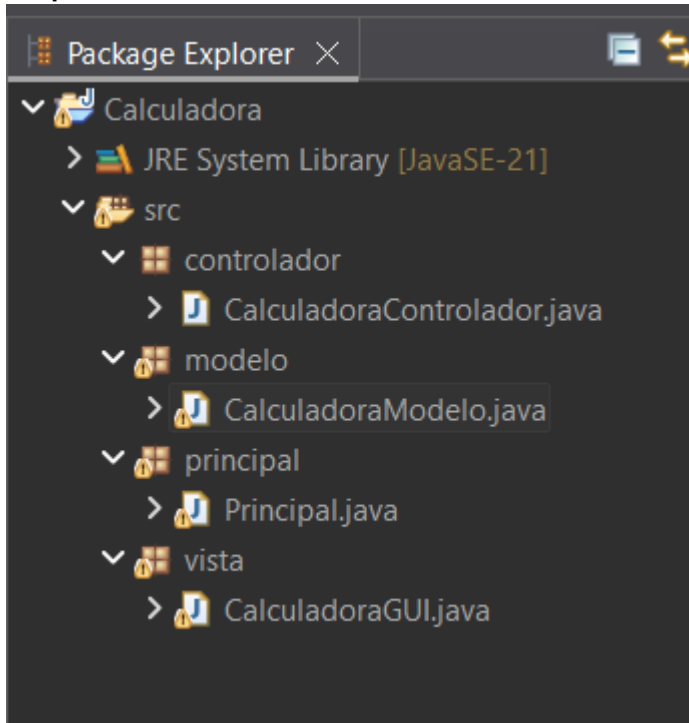
DIAGRAMA DE SECUENCIA – OPERACIÓN DE SUMA



	Dirección Académica	Código: CPE-FO-02-03
		Revisión: 1
	MANUAL DE PRÁCTICAS	Página: 13 de 27

IMPLEMENTACIÓN

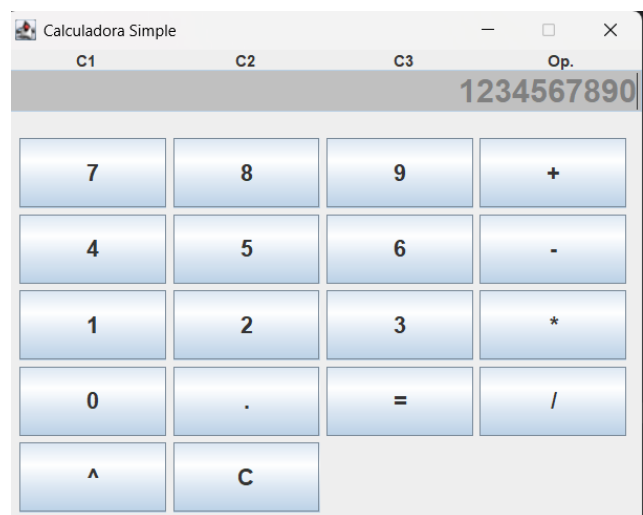
Carpeta Estructurado



CAPTURA DE LA INTERFAZ

Descripción:

- Ventana de 500x400 píxeles con título "Calculadora Simple"
- Campo de texto con fondo gris y texto inicial "1234567890"
- 16 botones distribuidos en grid de 5x4
- Etiquetas C1, C2, C3, Op. en la parte superior
- Botones: números 0-9, operadores (+, -, *, /, ^), =, C



	Dirección Académica	Código: CPE-FO-02-03
		Revisión: 1
	MANUAL DE PRÁCTICAS	Página: 14 de 27

	Dirección Académica	Código: CPE-FO-02-03
		Revisión: 1
	MANUAL DE PRÁCTICAS	Página: 15 de 27

“Controlador”

```

1 package controlador;
2
3 import modelo.CalculadoraModelo;
4 import vista.CalculadoraGUI;
5 import java.awt.event.ActionEvent;
6 import java.awt.event.ActionListener;
7 import java.math.BigInteger;
8
9 public class CalculadoraControlador implements ActionListener {
10
11     private CalculadoraModelo modelo;
12     private CalculadoraGUI vista;
13
14     public CalculadoraControlador(CalculadoraModelo modelo, CalculadoraGUI vista) {
15         this.modelo = modelo;
16         this.vista = vista;
17
18         this.vista.agregarActionListener(this);
19     }
20
21     @Override
22     public void actionPerformed(ActionEvent e) {
23         String comando = e.getActionCommand();
24         String textoActual = vista.getTextoPantalla();
25
26         try {
27             if (comando.matches("[0-9]")) {
28                 String nuevoTexto = modelo.agregarDigito(comando, textoActual);
29                 vista.setTextoPantalla(nuevoTexto);
30             } else if (comando.matches("[+\\-*/^]")) {
31
32                 if (!textoActual.contains("Error")) {
33                     modelo.setPrimerOperando(new BigInteger(textoActual));
34                     modelo.setOperador(comando);
35                     modelo.setNuevoNumero(true);
36                 }
37             }
38

```

	Dirección Académica	Código: CPE-FO-02-03
		Revisión: 1
	MANUAL DE PRÁCTICAS	Página: 16 de 27

```

38
39  } else if (comando.equals("=")) {
40
41  if (!textoActual.contains("Error") && !modelo.isNuevoNumero()) {
42      modelo.setSegundoOperando(new BigInteger(textoActual));
43      String resultado = modelo.realizarOperacion();
44      vista.setTextoPantalla(resultado);
45      modelo.setNuevoNumero(true);
46  }
47
48  } else if (comando.equals("C")) {
49      modelo = new CalculadoraModelo();
50      vista.setTextoPantalla("0");
51
52  } else if (comando.equals(".")) {
53
54      if (!textoActual.contains(".")) {
55          vista.setTextoPantalla(textoActual + ".");
56      }
57  }
58
59  } catch (NumberFormatException ex) {
60      vista.setTextoPantalla("Error");
61      modelo.setNuevoNumero(true);
62  } catch (ArithmeticException ex) {
63      vista.setTextoPantalla("Error: Div/0");
64      modelo.setNuevoNumero(true);
65  } catch (Exception ex) {
66      vista.setTextoPantalla("Error");
67      modelo.setNuevoNumero(true);
68  }
69  }
70  }

```

“Modelo”

	Dirección Académica	Código: CPE-FO-02-03
		Revisión: 1
	MANUAL DE PRÁCTICAS	Página: 17 de 27

```

1 package modelo;
2
3 import java.math.BigInteger;
4
5 public class CalculadoraModelo {
6     private BigInteger primerOperando;
7     private BigInteger segundoOperando;
8     private String operador;
9     private boolean nuevoNumero;
10    private static final BigInteger LIMITE_MAXIMO = new BigInteger("9999999999999999999");
11    private static final BigInteger LIMITE_MINIMO = new BigInteger("9999999999999999999");
12
13    public CalculadoraModelo() {
14        primerOperando = BigInteger.ZERO;
15        segundoOperando = BigInteger.ZERO;
16        operador = "";
17        nuevoNumero = true;
18    }
19
20    public void setPrimerOperando(BigInteger valor) {
21        this.primerOperando = valor;
22    }
23
24    public BigInteger getPrimerOperando() {
25        return primerOperando;
26    }
27
28    public void setSegundoOperando(BigInteger valor) {
29        this.segundoOperando = valor;
30    }
31
32    public BigInteger getSegundoOperando() {
33        return segundoOperando;
34    }
35
36    }
37
38    public void setOperador(String operador) {
39        this.operador = operador;
40    }

```

	Dirección Académica	Código: CPE-FO-02-03
		Revisión: 1
	MANUAL DE PRÁCTICAS	Página: 18 de 27

```

40
41 }
42
43 public String getOperador() {
44     return operador;
45 }
46
47
48 public void setNuevoNumero(boolean nuevoNumero) {
49     this.nuevoNumero = nuevoNumero;
50 }
51
52
53 public boolean isNuevoNumero() {
54     return nuevoNumero;
55 }
56 }
57

```



```
58 public String realizarOperacion() {
59     try {
60         BigInteger resultado = BigInteger.ZERO;
61
62         switch (operador) {
63             case "+":
64                 resultado = primerOperando.add(segundoOperando);
65                 break;
66             case "-":
67                 resultado = primerOperando.subtract(segundoOperando);
68                 break;
69             case "*":
70                 resultado = primerOperando.multiply(segundoOperando);
71                 break;
72             case "/":
73                 if (segundoOperando.equals(BigInteger.ZERO)) {
74                     return "Error: Div/0";
75                 }
76                 resultado = primerOperando.divide(segundoOperando);
77                 break;
78             case "^":
79                 resultado = primerOperando.pow(segundoOperando.intValue());
80                 break;
81             default:
82                 return "0";
83         }
84
85         if (resultado.abs().toString().length() > 20) {
86             return "Error: Desborde";
87         }
88
89         primerOperando = resultado;
90         return resultado.toString();
91     } catch (Exception e) {
92         return "Error";
93     }
94 }
95 }
```

	Dirección Académica	Código: CPE-FO-02-03
		Revisión: 1
	MANUAL DE PRÁCTICAS	Página: 20 de 27

```

96
97●   public String agregarDigito(String digito, String textoActual) {
98●       if (textoActual.equals("0") || textoActual.contains("Error") || nuevoNumero) {
99           nuevoNumero = false;
100          return digito;
101●       } else {
102          String nuevoTexto = textoActual + digito;
103●          if (nuevoTexto.length() > 20) {
104              return "Error: Desborde";
105          }
106          return nuevoTexto;
107      }
108  }
109
110●   public boolean validarNumero(String numero) {
111●       try {
112          BigInteger num = new BigInteger(numero);
113          return num.abs().toString().length() <= 20;
114●       } catch (Exception e) {
115          return false;
116      }
117  }
118 }

```

“Principal”

```

1  package principal;
2
3● import modelo.CalculadoraModelo;
4  import vista.CalculadoraGUI;
5  import controlador.CalculadoraControlador;
6  import javax.swing.SwingUtilities;
7
8  public class Principal {
9●   public static void main(String[] args) {
10
11       SwingUtilities.invokeLater(new Runnable() {
12           @Override
13●          public void run() {
14●              try {
15                  CalculadoraModelo modelo = new CalculadoraModelo();
16                  CalculadoraGUI vista = new CalculadoraGUI("Calculadora Simple");
17●                  CalculadoraControlador controlador = new CalculadoraControlador(modelo, vista);
18
19                  vista.setVisible(true);
20
21●              } catch (Exception e) {
22                  e.printStackTrace();
23              }
24          }
25      });
26  }
27 }

```

	Dirección Académica	Código: CPE-FO-02-03
		Revisión: 1
	MANUAL DE PRÁCTICAS	Página: 21 de 27

“Vista”

```

1 package vista;
2
3 import javax.swing.*;
4 import java.awt.*;
5 import java.awt.event.ActionListener;
6
7 public class CalculadoraGUI extends JFrame {
8
9     private JTextField txtPantalla;
10    private JButton[] btnNumeros;
11    private JButton btnSuma, btnResta, btnMultiplicacion, btnDivision, btnPotencia;
12    private JButton btnIgual, btnPunto, btnBorrar;
13
14    private String[] textoBotones = {
15        "7", "8", "9", "+",
16        "4", "5", "6", "-",
17        "1", "2", "3", "*",
18        "0", ".", "=", "/",
19        "^", "C"
20    };
21
22    public CalculadoraGUI(String nombre) {
23        super(nombre);
24        inicializarComponentes();
25        configurarVentana();
26    }
27
28    private void inicializarComponentes() {
29
30        setLayout(new BorderLayout(10, 10));
31
32        txtPantalla = new JTextField("1234567890");
33        txtPantalla.setHorizontalAlignment(JTextField.RIGHT);
34        txtPantalla.setEditable(false);
35        txtPantalla.setBackground(Color.LIGHT_GRAY);
36        txtPantalla.setFont(new Font("Arial", Font.BOLD, 24));
37        txtPantalla.setForeground(Color.GRAY);
38
39    }

```



```
40 JPanel panelBotones = new JPanel(new GridLayout(5, 4, 5, 5));
41 panelBotones.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
42
43 JPanel panelEtiquetas = new JPanel(new GridLayout(1, 4));
44 panelEtiquetas.add(new JLabel("C1", SwingConstants.CENTER));
45 panelEtiquetas.add(new JLabel("C2", SwingConstants.CENTER));
46 panelEtiquetas.add(new JLabel("C3", SwingConstants.CENTER));
47 panelEtiquetas.add(new JLabel("Op.", SwingConstants.CENTER));
48
49 JPanel panelSuperior = new JPanel(new BorderLayout());
50 panelSuperior.add(panelEtiquetas, BorderLayout.NORTH);
51 panelSuperior.add(txtPantalla, BorderLayout.CENTER);
52
53 btnNumeros = new JButton[10];
54
55 for (int i = 0; i < textoBotones.length; i++) {
56     JButton boton = new JButton(textoBotones[i]);
57     boton.setFont(new Font("Arial", Font.BOLD, 18));
58     boton.setFocusPainted(false);
59
60     if (textoBotones[i].matches("[0-9]")) {
61         int numero = Integer.parseInt(textoBotones[i]);
62         btnNumeros[numero] = boton;
63     } else {
64         switch (textoBotones[i]) {
65             case "+": btnSuma = boton; break;
66             case "-": btnResta = boton; break;
67             case "*": btnMultiplicacion = boton; break;
68             case "/": btnDivision = boton; break;
69             case "^": btnPotencia = boton; break;
70             case "=": btnIgual = boton; break;
71             case ".": btnPunto = boton; break;
72             case "C": btnBorrar = boton; break;
73         }
74     }
75     panelBotones.add(boton);
76 }
77
78
```

	Dirección Académica	Código: CPE-FO-02-03
		Revisión: 1
	MANUAL DE PRÁCTICAS	Página: 23 de 27

```

79         add(panelSuperior, BorderLayout.NORTH);
80         add(panelBotones, BorderLayout.CENTER);
81     }
82
83     private void configurarVentana() {
84         setSize(500, 400);
85         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
86         setLocationRelativeTo(null);
87         setResizable(false);
88     }
89
90     public String getTextoPantalla() {
91         return txtPantalla.getText();
92     }
93
94     public void setTextoPantalla(String texto) {
95         txtPantalla.setText(texto);
96     }
97
98     public void agregarActionListener(ActionListener listener) {
99         for (JButton boton : btnNumeros) {
100             if (boton != null) {
101                 boton.addActionListener(listener);
102             }
103         }
104
105         if (btnSuma != null) btnSuma.addActionListener(listener);
106         if (btnResta != null) btnResta.addActionListener(listener);
107         if (btnMultiplicacion != null) btnMultiplicacion.addActionListener(listener);
108         if (btnDivision != null) btnDivision.addActionListener(listener);
109         if (btnPotencia != null) btnPotencia.addActionListener(listener);
110         if (btnIgual != null) btnIgual.addActionListener(listener);
111         if (btnPunto != null) btnPunto.addActionListener(listener);
112         if (btnBorrar != null) btnBorrar.addActionListener(listener);
113     }

```

```

114
115     public void dataToUserInterface() {
116
117     }
118
119     public void userInterfaceToData() {
120
121     }
122 }

```

	Dirección Académica	Código: CPE-FO-02-03
		Revisión: 1
	MANUAL DE PRÁCTICAS	Página: 24 de 27

Nombre de la Prueba	Descripción	Datos de Entrada	Datos de salida
Suma básica	Sumar dos números positivos	$5 + 3$	8
Resta con negativos	Restar número mayor de menor	$3 - 8$	-5
Multiplicación	Multiplicar números	$7 * 6$	42
División exacta	División sin residuo	$15 / 3$	5
División entre cero	Caso de error	$5 / 0$	"Error: Div/0"
Desborde de dígitos	Exceder 20 dígitos	999... (21 dígitos)	"Error: Desborde"
Potencia	Calcular potencia	$2 ^ 5$	32
Operación encadenada	Múltiples operaciones	$5 + 3 * 2$	16 (sin prioridad)

Las pruebas demostraron que la calculadora:

- Maneja correctamente números enteros hasta 20 dígitos
- Detecta y muestra errores de división entre cero
- Detecta y muestra errores de desborde
- Realiza las operaciones básicas correctamente

	Dirección Académica	Código: CPE-FO-02-03
		Revisión: 1
	MANUAL DE PRÁCTICAS	Página: 25 de 27

CONCLUSIONES

- **Programación Orientada a Eventos:** Se comprendió cómo los eventos del usuario (clic en botones) determinan el flujo del programa.
- **Patrón MVC:** Se demostró la utilidad de separar la lógica de negocio de la interfaz de usuario, facilitando el mantenimiento y las pruebas.
- **Manejo de Errores:** Se implementaron validaciones para casos límite como desborde de memoria y división entre cero.
- **UML:** Los diagramas ayudaron a visualizar la arquitectura antes de la implementación, facilitando el desarrollo.
- **Java Swing:** Se adquirió experiencia en el desarrollo de interfaces gráficas con componentes como JFrame, JButton, JTextField y layouts.

- EVALUACIÓN Y RESULTADOS

Se describe la forma de evaluar la práctica desarrollada mediante la entrega del Informe Técnico solicitado por el Profesor, el cual puede contener tablas, planos, prototipos, gráficas, diagramas o dibujos, observaciones, conclusiones, cuestionario y referencias.

-REFERENCIAS

- Deitel, P. & Deitel, H. (2016). *Java How to Program* (10th ed.). Pearson.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Oracle Corporation. (2023). *The Java™ Tutorials: Creating a GUI With Swing*. <https://docs.oracle.com/javase/tutorial/uiswing/>

	Dirección Académica	Código: CPE-FO-02-03
		Revisión: 1
	MANUAL DE PRÁCTICAS	Página: 26 de 27

-ANEXOS

Se recomienda poner en anexos las partes del trabajo que, intercaladas en medio del texto romperían la continuidad de lectura del mismo, por ejemplo deducciones detalladas de alguna expresión, cálculos largos y detallados, códigos y enumeración detallada de algunos componentes. El docente según considere podría proporcionar tablas de valores, indicadores, manuales de equipos u otros.